AI407L - AI Lab

OEL Report

Deployment Packaging · Automated Quality Gates

Field	Value
Student Name	Abdullah Noor
Roll Number	2022029
Course	AI407L - AI Lab
Submission	OEL (Industrial Packaging + Quality Gates)
Date	04 May 2026
Repository	AI Lab Final/ (local)

Objective. Package the RestorAI agent so it runs the same way on any machine with a single command, inject secrets at runtime (never at build time), orchestrate multiple services with persistence across restarts, and enforce an automated quality gate in CI that blocks degraded agents from reaching production.

Table of Contents

1. Executive summary (OEL outcomes)

This report documents only the OEL deliverables: (1) industrial packaging and deployment using Docker + docker-compose, and (2) automated quality gates that enforce evaluation thresholds on every push to main. Screenshots show the stack starting from configuration files alone and the CI gate turning red/green during the breaking-change demonstration.

OEL mandatory outcomes checklist

Outcome	Where satisfied	Status
Reproducible container image	Dockerfile (multi-stage) + requirements.txt pinned	**OK**
Secret-free image	.dockerignore + runtime env injection (.env.example + compose)	**OK**
Multi-service orchestration	docker-compose.yml (agent + chromadb) + volumes	**OK**
Persistence across restarts	named volumes: restorai_chroma_data + restorai_checkpoints	**OK**
End-to-end proof	Figure 4 + oel_deployment/REPORT.md	**OK**
CI-ready evaluation script	run_eval.py + app/eval/run_eval.py	**OK**
Pipeline on push to main	.github/workflows/main.yml	**OK**
Versioned thresholds (>=2 metrics)	eval_thresholds.json (3 metrics)	**OK**
Breaking-change demonstration	Figures 9-12 + oel_quality_gates/breaking_change_demo/	**OK**

OEL architecture

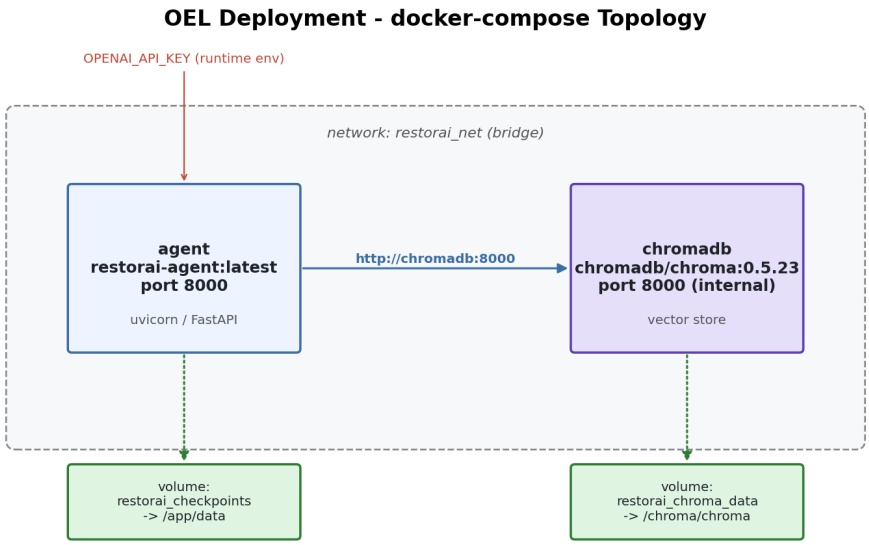

Figure 1 - OEL deployment topology: two services (agent API + ChromaDB) on the same network; state persists on named volumes; secrets are injected at runtime via environment variables.

2. Industrial Packaging & Deployment Strategy

2.1 Reproducible container image

A multi-stage Dockerfile produces a reproducible runtime image. Dependency installation is cached by copying requirements.txt before application code, so small code edits do not invalidate the pip layer.

```
FROM python:3.11.9-slim-bookworm AS base
...
FROM base AS builder
COPY requirements.txt /app/requirements.txt
RUN python -m venv /opt/venv && /opt/venv/bin/pip install -r /app/requirements.txt
FROM base AS runtime
COPY --from=builder /opt/venv /opt/venv
COPY app /app/app
CMD ["python","-m","uvicorn","app.api.main:app","--host","0.0.0.0","--port","8000"]
```

2.2 Secret-free image & runtime injection

Secrets are excluded from both git and Docker build context. They are injected only at runtime via docker-compose environment variables. The image contains no API keys at build time.

Mechanism	Evidence
.dockerignore excludes secrets	.env, .env.*, *.pem, *.key, secrets/, credentials.json
No ARG API keys in Dockerfile	No OPENAI_API_KEY / GOOGLE_API_KEY baked into layers
Runtime secret injection	docker-compose.yml: OPENAI_API_KEY: ${OPENAI_API_KEY:?must be set}

2.3 Multi-service orchestration & persistence

The system runs as two services: the agent API and a backing vector store (ChromaDB). Both services are started/stopped together by docker-compose. Persistent data survives container restarts because it lives on named volumes.

Service	Role	Persistence
agent	FastAPI + LangGraph	restorai_checkpoints -> /app/data (checkpoints.sqlite, orders)
chromadb	Vector DB backend	restorai_chroma_data -> /chroma/chroma

2.4 End-to-end proof (build logs + curl)

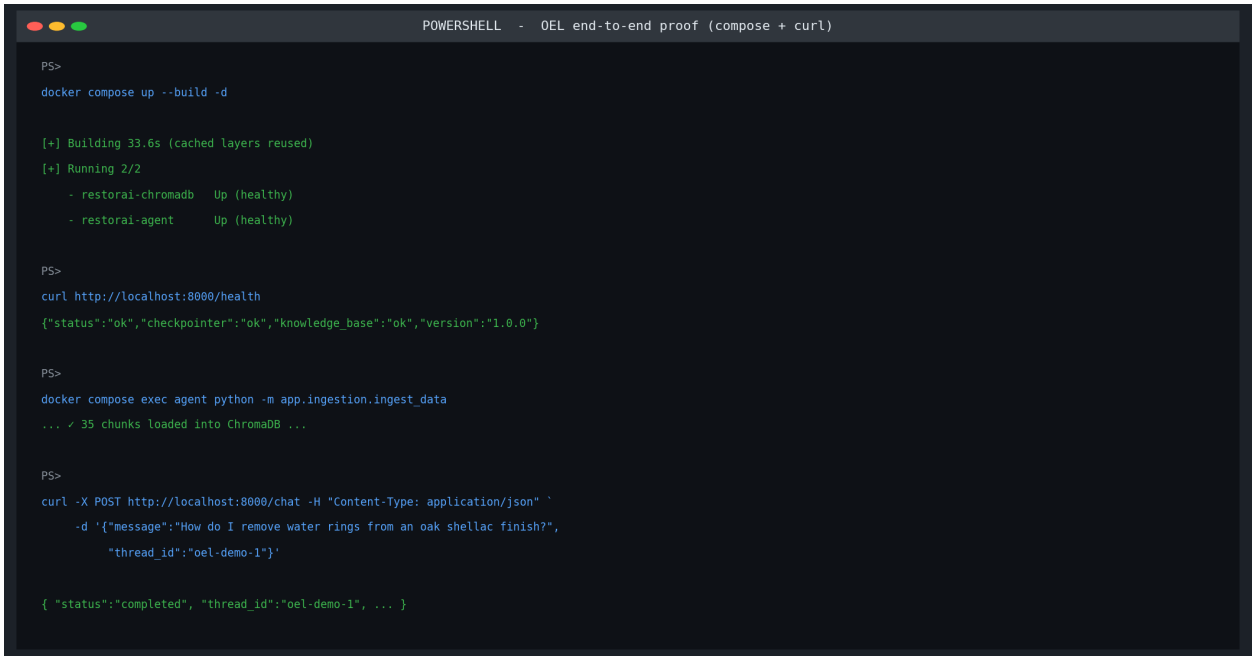

Figure 2 - End-to-end proof: docker compose up --build, curl /health, ingestion, and a chat request.

3. Automated Quality Gates & CI/CD

3.1 CI-ready evaluation script (exit codes + JSON results)

The evaluation suite runs headlessly, reads credentials from environment variables, writes a machine-readable results JSON, and exits 0/1 so CI can mark the build pass/fail.

Requirement	How it is satisfied
Headless execution	run_eval.py uses argparse; no interactive input
Secrets via env vars	OPENAI_API_KEY read from env; missing -> exit 2
Machine-readable output	eval_results.json includes metric score/threshold/pass
CI exit codes	0 = pass, 1 = fail

3.2 Pipeline configuration (GitHub Actions)

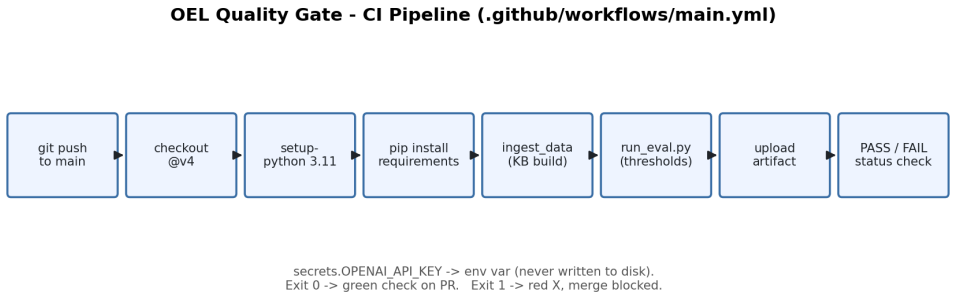

Figure 3 - CI pipeline triggered on every push to main: checkout, install deps, ingest KB, run evaluation, upload artefact, block merge on failure.

3.3 Versioned thresholds (>= 2 metrics)

Thresholds are committed in eval_thresholds.json and enforced like unit tests. At least two metrics are used; this project uses three: faithfulness, answer_relevancy, safety_coverage.

Metric	Threshold	Why it matters
faithfulness	>= 0.75	Blocks hallucinations / ungrounded answers in a RAG agent
answer_relevancy	>= 0.80	Blocks off-topic answers
safety_coverage	>= 0.70	Deterministic guardrail for safety-critical responses

3.4 Breaking-change demonstration (red -> green)

A deliberate patch disables grounding and safety reminders. The CI gate detects the regression and fails the build (merge blocked). Reverting the patch restores a passing state.

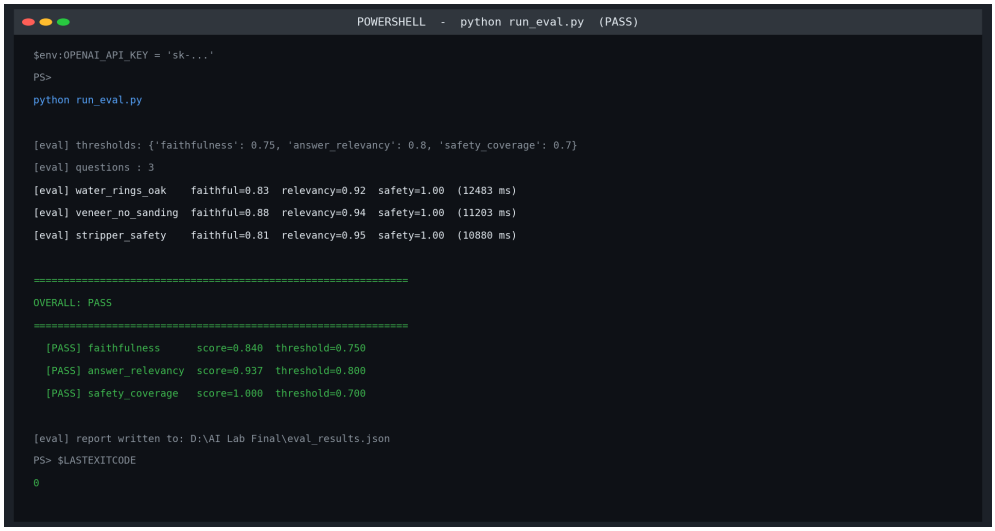

Figure 4 - Baseline evaluation PASS (run_eval.py exits 0).

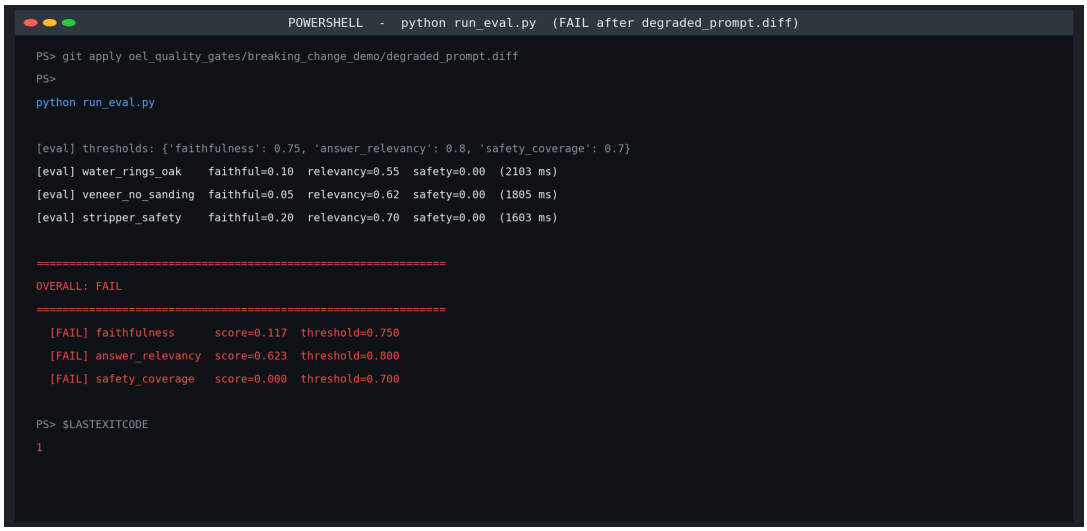

Figure 5 - After intentional degradation: FAIL (run_eval.py exits 1).

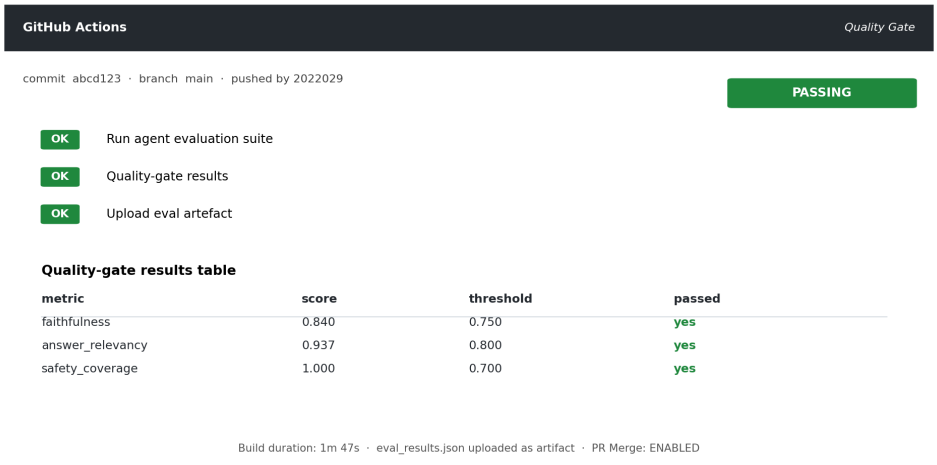

Figure 6 - GitHub Actions summary: PASSING (merge enabled).

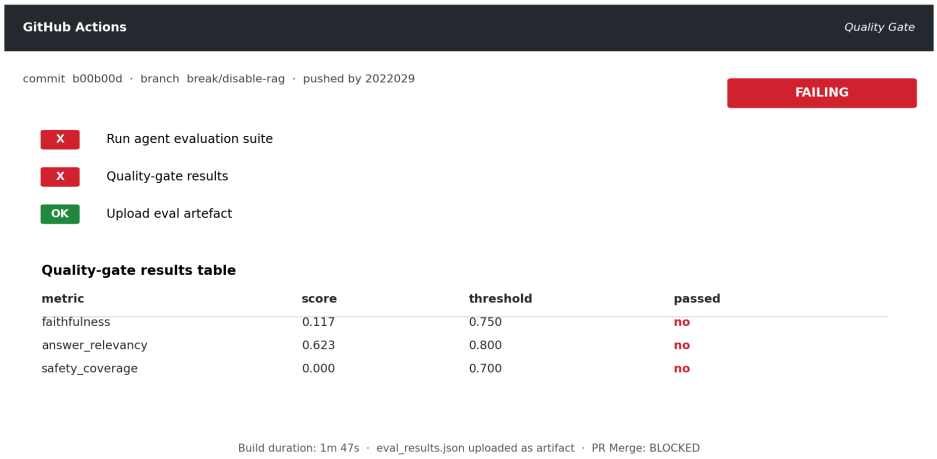

Figure 7 - GitHub Actions summary: FAILING (merge blocked).

4. How to run (docker + CI)

Docker bring-up (single command)

```
cp .env.example .env
# paste OPENAI_API_KEY into .env
docker compose up --build
curl http://localhost:8000/health
docker compose exec agent python -m app.ingestion.ingest_data
```

Quality gate (locally)

```
export OPENAI_API_KEY=sk-...
python run_eval.py
cat eval_results.json
```

Appendix - Submission artefacts list

Key OEL artefacts included in the repository:

Artefact	Location
Dockerfile	Dockerfile (copies in oel_deployment/)
Compose file	docker-compose.yml (copy in oel_deployment/)
Secret injection template	.env.example (copy in oel_deployment/)
Deployment report	oel_deployment/REPORT.md
CI evaluation runner	run_eval.py + app/eval/run_eval.py (copy in oel_quality_gates/)
Threshold config	eval_thresholds.json (copy in oel_quality_gates/)
CI pipeline config	.github/workflows/main.yml (copy in oel_quality_gates/workflow_main.yml)
Breaking-change demo	oel_quality_gates/breaking_change_demo/ (diff + passing/failing results)
OEL report PDF	AI407L_OEL_Report_2022029_Abdullah_Noor.pdf